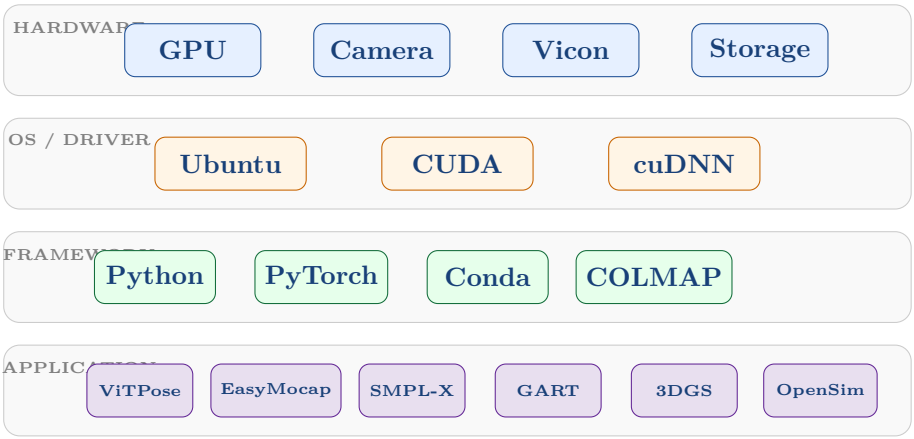
GART-Pitch Development Guide

GART-Pitch
개발 환경 세팅 가이드

하드웨어, 소프트웨어, 데이터 파이프라인
A-to-Z 실전 구축 매뉴얼



2026

Part 5 연구 제안서(GART-Pitch) 구현을 위한
하드웨어·소프트웨어·데이터 환경 구축 실전 가이드.

Contents

1 전체 아키텍처 개요 **2**
1.1 GART-Pitch 파이프라인 구성요소 2

2 하드웨어 요구사항 **3**
2.1 컴퓨팅 하드웨어 3
2.2 촬영 하드웨어 4
2.3 데이터 저장량 산정 4

3 소프트웨어 환경 구축 **5**
3.1 운영체제 및 드라이버 5
3.2 Python 환경 (Conda) 6
3.3 핵심 Python 패키지 7

4 핵심 도구 설치 **8**
4.1 COLMAP (카메라 보정 + SfM) 8
4.2 ViTPose / MMPose (2D 자세 추정) 9
4.3 SMPL-X 모델 10
4.4 EasyMocap (다시점 SMPL-X 피팅) 10
4.5 3D Gaussian Splatting (원본 구현) 11
4.6 GART (핵심 프레임워크) 11
4.7 OpenSim (생체역학) 12
4.8 ffmpeg (영상 처리) 12

5 프로젝트 디렉토리 구조 **13**

6 환경 통합 테스트 **16**
6.1 단계별 검증 체크리스트 16
6.2 통합 테스트 스크립트 17

7 의존성 파일 **18**
7.1 requirements.txt 19
7.2 conda environment.yml (대안) 20

8 설치 순서 요약 (Quick Start) **21**

A 트러블슈팅 **23**
A.1 흔한 설치 오류와 해결 23

B 참고 리포지토리 목록 **24**

Chapter 1

전체 아키텍처 개요

1.1 GART-Pitch 파이프라인 구성요소

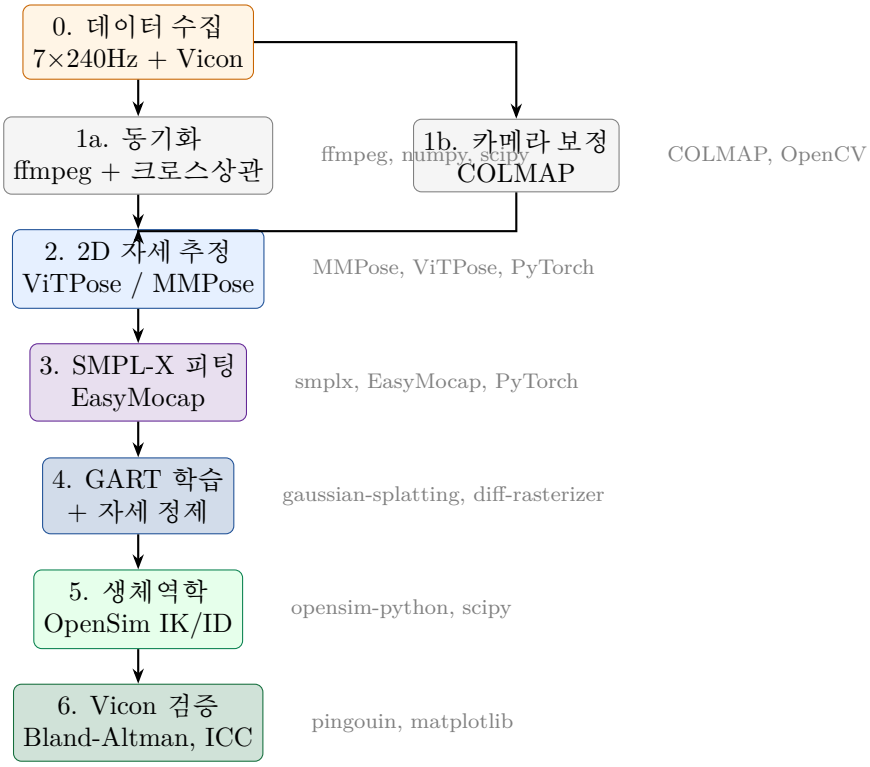


Figure 1.1: GART-Pitch 개발 파이프라인과 각 단계별 핵심 도구

Key Idea

전체 파이프라인은 7개 단계, 약 15개의 핵심 소프트웨어 컴포넌트로 구성된다. 각 단계는 독립적으로 테스트 가능하며, 순차적으로 구축하는 것을 권장한다. 가장 까다로운 설치는 **Stage 4 (GART/3DGS)**의 CUDA 커스텀 래스터라이저이다.

Chapter 2

하드웨어 요구사항

2.1 컴퓨팅 하드웨어

Table 2.1: 컴퓨팅 하드웨어 사양 (최소 / 권장 / 최적)

항목	최소	권장	최적
GPU	RTX 3090 (24GB)	RTX 4090 (24GB)	A100 80GB / H100
GPU VRAM	24 GB	24 GB	48–80 GB
CPU	8코어	16코어+ (i9/Ryzen 9)	32 코어 + (Threadripper)
RAM	32 GB	64 GB	128 GB
스토리지	1 TB NVMe SSD	2 TB NVMe SSD	4 TB+ NVMe SSD
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS	Ubuntu 22.04/24.04

Warning

GPU VRAM이 가장 중요한 병목이다. GART는 프레임당 100K–500K Gaussian을 처리하며, 7대 카메라 1080p 렌더링 시 피크 VRAM ∼18–22 GB가 필요하다. 24GB 미만에서는 배치 크기 축소 또는 해상도 다운샘플링이 필요하다.

2.2 촬영 하드웨어

Table 2.2: 촬영 장비 사양

장비	사양
RGB 카메라 ×7	240 Hz 이상, 1920×1080. 권장: GoPro Hero 12 (240fps), Sony RX0 II, 또는 산업용 카메라 (FLIR Blackfly S). 노출시간: ≤ 1/1000초 (모션블러 방지).
Vicon 시스템	12대+ 적외선 카메라, 250–500 Hz. Vicon Vantage/Vero 또는 동급. 반사 마커 39–59개 (상지 강화 마커세트).
동기화 장치	Vicon Lock (하드웨어 genlock) 또는 오디오 기반 소프트웨어 동기화. 시간 정밀도: < 1 ms 필요.
보정 도구	ChArUco 보드 (A2 이상). Vicon 마커 + 체커보드 코너 겸용. 정적 보정용 + 동적 보정용 각 1매.
조명	1000+ lux 균일 조명 (LED 패널 권장). 고속 촬영에서 짧은 노출 보상 필수. 형광등 플리커 주의 (50/60Hz).
저장 장치	촬영용: 카메라별 V30+ microSD 256GB 이상. 아카이브: 10명 연구 기준 ProRes ∼3 TB, H.265 ∼90 GB.

2.3 데이터 저장량 산정

Table 2.3: 포맷별 데이터 저장량 (7대 카메라 기준)

포맷	투구 1회 (2초)	세션 (100투)	10명 연구	품질
RAW	∼20 GB	∼2 TB	∼20 TB	무손실
ProRes 422 HQ	2–3 GB	200–300 GB	2–3 TB	거의 무손실
H.264 (고품질)	100–175 MB	10–17.5 GB	100–175 GB	약간의 손실
H.265 (고품질)	50–90 MB	5–9 GB	50–90 GB	약간의 손실

Important

3DGS 학습에서는 **광도 손실(photometric loss)**이 목적함수이므로 압축 아티팩트가 노이즈로 작용한다. **분석용 아카이브는 ProRes 이상**, 3DGS 학습용은 **고품질 H.265**가 실용적 절충안이다.

Chapter 3

소프트웨어 환경 구축

3.1 운영체제 및 드라이버

Table 3.1: 기본 시스템 요구사항

항목	버전	비고
OS	Ubuntu 22.04 LTS	24.04도 호환; macOS는 전처리만 가능
NVIDIA Driver	$\geq$ 535	`nvidia-smi`로 확인
CUDA Toolkit	11.8 또는 12.1	PyTorch 빌드 버전과 일치 필수
cuDNN	$\geq$ 8.9	CUDA 버전에 맞는 것
GCC / G++	11.x	CUDA 커스텀 커널 빌드에 필요

NVIDIA 드라이버 + CUDA 설치

```
# 1. NVIDIA 드라이버최신 ( 안정버전 )
sudo apt update && sudo apt install -y nvidia-driver-535
sudo reboot

# 2. CUDA 11.8 (runfile 방식 - 가장확실 )
wget https://developer.download.nvidia.com/compute/cuda/11.8.0/
  local_installers/cuda_11.8.0_520.61.05_linux.run
sudo sh cuda_11.8.0_520.61.05_linux.run --toolkit --silent

# 3. 환경변수설정  (~/.bashrc)
export CUDA_HOME=/usr/local/cuda-11.8
export PATH=$CUDA_HOME/bin:$PATH
export LD_LIBRARY_PATH=$CUDA_HOME/lib64:$LD_LIBRARY_PATH

# 4. 확인
nvidia-smi            # 드라이버및  GPU 확인
nvcc --version        # CUDA 버전확인
```

3.2 Python 환경 (Conda)

Conda 환경 생성

```
# Miniconda 설치이미 ( 설치되어있으면생략 )
wget https://repo.anaconda.com/miniconda/
  Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh -b -p ~/miniconda3
eval "$(~/miniconda3/bin/conda shell.bash hook)"

# GART-Pitch 전용환경생성
conda create -n gart-pitch python=3.10 -y
conda activate gart-pitch

# PyTorch (CUDA 11.8 빌드)
pip install torch==2.1.0 torchvision==0.16.0 \
    --index-url https://download.pytorch.org/whl/cu118

# GPU 확인
python -c "import torch; print(torch.cuda.is_available())"
python -c "import torch; print(torch.cuda.get_device_name(0))"
```

Warning

PyTorch와 CUDA 버전 불일치가 가장 흔한 설치 실패 원인이다. `nvcc --version`의 CUDA와 `torch.version.cuda`가 반드시 일치해야 한다. CUDA 12.1을 사용할 경우 `cu121` 빌드를 설치한다.

3.3 핵심 Python 패키지

Table 3.2: 핵심 Python 의존성

패키지	버전	용도	설치
— 기본 과학 계산 —			
numpy	≥1.24	수치 연산	pip
scipy	≥1.11	신호처리, 최적화, 회전	pip
opencv-python	≥4.8	영상처리, 보정	pip
matplotlib	≥3.7	시각화	pip
— 3D 모델링 —			
smplx	0.1.28+	SMPL-X 인체 모델	pip
trimesh	≥3.23	3D 메시 처리	pip
pyrender	0.1.45+	3D 렌더링 시각화	pip
open3d	≥0.17	점군 시각화/처리	pip
— 생체역학 —			
opensim	4.5+	역운동학/역동역학	conda-forge
pingouin	≥0.5.3	ICC, Bland-Altman	pip
— 기타 —			
tqdm	—	진행률 표시	pip
tensorboard	—	학습 모니터링	pip
wandb	—	실험 관리 (선택)	pip

기본 패키지 일괄 설치

```
conda activate gart-pitch

# 기본과학계산
pip install numpy scipy opencv-python matplotlib

# 3D 모델링
pip install smplx trimesh pyrender open3d

# 생체역학
conda install -c opensim-org opensim=4.5 -y   # 또는 pip
pip install pingouin

# 기타
pip install tqdm tensorboard
```

Chapter 4

핵심 도구 설치

4.1 COLMAP (카메라 보정 + SfM)

COLMAP 설치

```
# 방법 1: apt (Ubuntu 22.04, GPU 지원없을수있음 )
sudo apt install -y colmap

# 방법 2: 소스빌드 (GPU 가속권장 )
sudo apt install -y cmake build-essential \
    libboost-all-dev libceres-dev \
    libfreeimage-dev libgoogle-glog-dev \
    libglew-dev libsqlite3-dev \
    qtbase5-dev libqt5opengl5-dev

git clone https://github.com/colmap/colmap.git
cd colmap && mkdir build && cd build
cmake .. -DCMAKE_CUDA_ARCHITECTURES=native
make -j$(nproc)
sudo make install

# 확인
colmap -h
```

COLMAP 설치 확인

- ☐ `colmap -h` 실행 시 도움말 출력
- ☐ `colmap feature_extractor` GPU 사용 가능
- ☐ 재투영 오차 $< 0.5\,\mathrm{px}$ 달성 가능

4.2 ViTPose / MMPose (2D 자세 추정)

MMPose + ViTPose 설치

```
# MMPose (ViTPose 포함)
pip install -U openmim
mim install mmengine
mim install "mmcv>=2.0.0"
mim install "mmdet>=3.0.0"
mim install "mmpose>=1.0.0"

# ViTPose-Huge 모델가중치다운로드
mkdir -p ~/models/vitpose
wget -P ~/models/vitpose/ \
  https://download.openmmlab.com/mmpose/v1/body_2d_keypoint/
  topdown_heatmap/coco/
  td-hm_ViTPose-huge_8xb64-210e_coco-256x192-e32adcd4_20230314.pth

# 테스트단일 ( 이미지)
python -c "
from mmpose.apis import MMPoseInferencer
inferencer = MMPoseInferencer(
    pose2d='td-hm_ViTPose-huge_8xb64-210e_coco-256x192')
print('ViTPose loaded successfully')
"
```

Important

240 Hz 고속 영상에서는 **ViTPose-Huge**를 권장한다. RTMPose는 실시간 추론에 유리하지만, 모션 블러가 심한 가속 단계에서는 ViTPose의 Transformer 기반 전역 문맥 활용이 더 강건하다.

4.3 SMPL-X 모델

SMPL-X 모델 다운로드 및 설정

```
# 1. SMPL-X 모델다운로드
#    https://smpl-x.is.tue.mpg.de 에서라이선스동의후다운로드
# 또는    Google 에서Drive 다운로드별도 ( 안내)

# 2. 디렉토리구조
mkdir -p ~/models/smplx
# 다운로드한파일배치  :
# ~/models/smplx/
#   SMPLX_NEUTRAL.npz
#   SMPLX_MALE.npz
#   SMPLX_FEMALE.npz

# 3. 환경변수설정  (~/.bashrc)
export SMPLX_MODEL_PATH=~/models/smplx

# 4. 테스트
python -c "
import smplx, torch
model = smplx.create('$HOME/models/smplx',
    model_type='smplx', gender='neutral')
output = model()
print(f'Vertices: {output.vertices.shape}')
print(f'Joints:   {output.joints.shape}')
"
# 예상출력 : Vertices: torch.Size([1, 10475, 3])
#            Joints:   torch.Size([1, 127, 3])
```

4.4 EasyMocap (다시점 SMPL-X 피팅)

EasyMocap 설치

```
git clone https://github.com/zju3dv/EasyMocap.git
cd EasyMocap
pip install -e .

# 의존성
pip install yacs chumpy

# 구조확인
python -c "import easymocap; print('EasyMocap OK')"
```

4.5 3D Gaussian Splatting (원본 구현)

3DGS + 미분 가능 래스터라이저 설치

```
# 1. 원본 3DGS 클론
git clone https://github.com/graphdeco-inria/
  gaussian-splatting.git --recursive
cd gaussian-splatting

# 2. 서브모듈 (diff-gaussian-rasterization)
pip install submodules/diff-gaussian-rasterization
pip install submodules/simple-knn

# 3. 나머지의존성
pip install plyfile

# 4. 테스트학습 ( 실행)
# python train.py -s 데이터경로<> (COLMAP 결과필요 )
```

Warning

diff-gaussian-rasterization은 **CUDA C++ 커스텀 커널**을 빌드한다. 빌드 실패의 주요 원인:

- CUDA 버전과 PyTorch CUDA 버전 불일치
- GCC 버전이 CUDA와 호환되지 않음 (CUDA 11.8 → GCC 11 권장)
- CUDA_HOME 환경변수 미설정

4.6 GART (핵심 프레임워크)

GART 설치

```
# GART 클론
git clone https://github.com/JiahuiLei/GART.git
cd GART

# 의존성설치
pip install -r requirements.txt

# 는GART 자체 diff-를rasterizer 사용할수있음
# 또는원본  3의DGS 것을공유
pip install ./submodules/diff-gaussian-rasterization  # 있는경우

# 테스트
python -c "
import sys; sys.path.insert(0, '.')
print('GART directory structure OK')
"
```

4.7 OpenSim (생체역학)

OpenSim Python 바인딩 설치

```
# 방법 1: conda-forge 권장()
conda install -c opensim-org opensim=4.5 -y

# 방법 2: pip 일부( 환경에서만가능 )
pip install opensim

# 테스트
python -c "
import opensim as osim
model = osim.Model()
model.setName('test')
print(f'OpenSim {osim.GetVersion()} loaded')
"
```

4.8 ffmpeg (영상 처리)

ffmpeg 설치

```
# Ubuntu
sudo apt install -y ffmpeg

# 프레임추출  (240fps -> 개별이미지 )
ffmpeg -i device1_synced.MOV -qscale:v 2 \
    frames/device1/frame_%06d.jpg

# 해상도코덱/ 확인
ffprobe -v error -select_streams v:0 \
    -show_entries stream=width,height,r_frame_rate,codec_name \
    -of csv=p=0 device1_synced.MOV
```

Chapter 5

프로젝트 디렉토리 구조

권장 디렉토리 레이아웃

```
gart-pitch/
|-- configs/                      # 실험설정  YAML
|   |-- default.yaml
|   |-- phase_adaptive.yaml       # 위상별정규화가중치
|
|-- data/
|   |-- raw/                      # 원본영상
|   |   |-- subject_01/
|   |   |   |-- set_001/
|   |   |   |   |-- device1_synced.MOV
|   |   |   |   |-- ...
|   |   |   |   |-- device7_synced.MOV
|   |   |   |   |-- sync_offsets.json
|   |   |   |   |-- _synced_meta.json
|   |
|   |-- frames/                   # 추출된프레임  (jpg/png)
|   |   |-- subject_01/set_001/
|   |       |-- cam1/ cam2/ ... cam7/
|   |
|   |-- calibration/
|   |   |-- intrinsics/           # 카메라별내부파라미터
|   |   |-- extrinsics/           # COLMAP sparse 결과
|   |   |-- colmap_workspace/     # COLMAP DB + sparse model
|   |
|   |-- vicon/                    # Vicon gold standard
|       |-- subject_01/
|           |-- set_001.c3d
|           |-- set_001.csv
|           |-- ik_results/       # OpenSim IK 결과
|
|-- models/                       # 사전학습모델
|   |-- smplx/                    # SMPLX_NEUTRAL.npz 등
|   |-- vitpose/                  # ViTPose-Huge 가중치
|   |-- opensim/                  # 근골격모델  (.osim)
|
|-- src/                          # 소스코드
|   |-- preprocessing/
|   |   |-- sync.py               # 오디오기반동기화
|   |   |-- extract_frames.py     # ffmpeg 프레임추출
|   |   |-- calibrate.py          # COLMAP 래퍼
|   |
|   |-- pose2d/
|   |   |-- run_vitpose.py        # 다시점 2D 자세추정
|   |
|   |-- smplx_fitting/
|   |   |-- run_easymocap.py      # EasyMocap 래퍼
|   |   |-- visualize_smplx.py    # SMPL-X 결과시각화
|   |
|   |-- gart/
|   |   |-- train_canonical.py    # 캐노니컬 Gaussian 학습
|   |   |-- refine_pose.py        # 렌더링기반자세정제
|   |   |-- phase_detector.py     # 투구위상자동검출
|   |   |-- losses.py             # 위상별적응형손실
|   |
|   |-- biomechanics/
|   |   |-- isb_decompose.py      # ISB 관절각분해
|   |   |-- virtual_markers.py    # 가상마커추출
|   |   |-- opensim_ik.py         # OpenSim IK 래퍼
|   |   |-- opensim_id.py         # OpenSim ID 래퍼
|   |
|   |-- validation/
```

Chapter 6

환경 통합 테스트

6.1 단계별 검증 체크리스트

Table 6.1: 환경 설치 완료 검증 체크리스트

#	체크	검증 명령어	기대 결과
1	☐	`nvidia-smi`	GPU 이름, 드라이버 버전 출력
2	☐	`nvcc --version`	CUDA 11.8 또는 12.1
3	☐	`python -c "import torch; print(torch.cuda.is_available())"`	`True`
4	☐	`python -c "import torch; print(torch.version.cuda)"`	11.8 또는 12.1
5	☐	`colmap -h`	도움말 출력
6	☐	`python -c "from mmpose.apis import MMPoseInferencer"`	에러 없음
7	☐	`python -c "import smplx"`	에러 없음
8	☐	SMPL-X 모델 로드 테스트	vertices: [1, 10475, 3]
9	☐	`python -c "import easymocap"`	에러 없음
10	☐	`diff-gaussian-rasterization` import	에러 없음
11	☐	`python -c "import opensim"`	버전 출력
12	☐	`ffmpeg -version`	버전 출력

6.2 통합 테스트 스크립트

전체 환경 검증 스크립트 (test_env.py)

```python
#!/usr/bin/env python3
"""GART-Pitch environment verification script."""
import sys

def check(name, test_fn):
    try:
        result = test_fn()
        print(f"  [OK] {name}: {result}")
        return True
    except Exception as e:
        print(f"  [FAIL] {name}: {e}")
        return False

print("=" * 60)
print("GART-Pitch Environment Verification")
print("=" * 60)
results = []

# 1. PyTorch + CUDA
results.append(check("PyTorch CUDA",
    lambda: f"torch={__import__('torch').__version__}, "
            f"cuda={__import__('torch').version.cuda}, "
            f"gpu={__import__('torch').cuda.get_device_name(0)}"))

# 2. Core packages
for pkg in ['numpy', 'scipy', 'cv2', 'matplotlib']:
    results.append(check(pkg,
        lambda p=pkg: __import__(p).__version__))

# 3. 3D packages
for pkg in ['smplx', 'trimesh', 'open3d']:
    results.append(check(pkg,
        lambda p=pkg: __import__(p).__version__))

# 4. GART deps
results.append(check("diff-gaussian-rasterization",
    lambda: __import__('diff_gaussian_rasterization')
            and "loaded"))

# 5. MMPose
results.append(check("mmpose",
    lambda: __import__('mmpose').__version__))

# 6. OpenSim
results.append(check("opensim",
    lambda: __import__('opensim').GetVersion()))

# Summary
passed = sum(results)
total = len(results)
print("=" * 60)
print(f"Result: {passed}/{total} passed")
if passed == total:
    print("All checks passed! Ready for GART-Pitch.")
else:
    print(f"WARNING: {total-passed} checks failed.")
print("=" * 60)
```

Chapter 7

의존성 파일

7.1 requirements.txt

requirements.txt

```
# ===== GART-Pitch requirements.txt =====
# Install: pip install -r requirements.txt
# Note: PyTorch must be installed separately with correct CUDA

# --- Core Scientific ---
numpy>=1.24
scipy>=1.11
opencv-python>=4.8
matplotlib>=3.7
Pillow>=9.5

# --- 3D Modeling ---
smplx>=0.1.28
trimesh>=3.23
pyrender>=0.1.45
open3d>=0.17
chumpy
plyfile

# --- Pose Estimation (install via mim separately) ---
# mim install mmengine mmcv>=2.0.0 mmdet>=3.0.0 mmpose>=1.0.0

# --- SMPL-X Fitting ---
# EasyMocap: pip install -e /path/to/EasyMocap
yacs

# --- 3D Gaussian Splatting ---
# diff-gaussian-rasterization: pip install from submodule
# simple-knn: pip install from submodule

# --- Biomechanics ---
# opensim: conda install -c opensim-org opensim=4.5
pingouin>=0.5.3

# --- Experiment Management ---
tqdm
tensorboard
pyyaml
```

7.2 conda environment.yml (대안)

environment.yml

```yml
name: gart-pitch
channels:
  - pytorch
  - nvidia
  - conda-forge
  - opensim-org
  - defaults
dependencies:
  - python=3.10
  - pytorch=2.1.0
  - torchvision=0.16.0
  - pytorch-cuda=11.8
  - numpy>=1.24
  - scipy>=1.11
  - matplotlib>=3.7
  - opencv>=4.8
  - opensim=4.5
  - pip:
    - smplx>=0.1.28
    - trimesh>=3.23
    - pyrender>=0.1.45
    - open3d>=0.17
    - pingouin>=0.5.3
    - tensorboard
    - tqdm
    - pyyaml
    - yacs
    - plyfile
    - chumpy
```

Chapter 8

설치 순서 요약 (Quick Start)

전체 설치 순서 (약 2–3 시간)

1. **NVIDIA 드라이버 설치** (10분)
   `sudo apt install nvidia-driver-535 && sudo reboot`
2. **CUDA 11.8 설치** (10분)
   runfile 다운로드 → `--toolkit --silent` → 환경변수 설정
3. **Conda 환경 생성** (5분)
   `conda create -n gart-pitch python=3.10`
4. **PyTorch 설치** (5분)
   `pip install torch==2.1.0 torchvision==0.16.0 --index-url .../cu118`
5. **기본 패키지 설치** (5분)
   `pip install numpy scipy opencv-python matplotlib smplx trimesh pyrender open3d`
6. **COLMAP 소스 빌드** (30분)
   git clone → cmake → make
7. **MMPose + ViTPose 설치** (10분)
   mim install mmengine mmcv mmdet mmpose → 모델 가중치 다운로드
8. **SMPL-X 모델 다운로드** (10분)
   라이선스 동의 → 다운로드 → `$SMPLX_MODEL_PATH` 설정
9. **EasyMocap 설치** (5분)
   git clone → `pip install -e .`
10. **3D Gaussian Splatting 빌드** (15분)
    git clone –recursive → pip install submodules/*
11. **GART 설치** (10분)
    git clone → pip install -r requirements.txt
12. **OpenSim 설치** (5분)
    `conda install -c opensim-org opensim=4.5`
13. **ffmpeg 설치** (2분)
    `sudo apt install ffmpeg`
14. **환경 검증** (5분)
    `python test_env.py` — 12개 항목 모두 OK 확인

Key Idea

가장 주의해야 할 3가지:

1. **CUDA 버전 일관성**: `nvcc`, PyTorch, diff-rasterizer 모두 동일 CUDA 버전
2. **SMPL-X 라이선스**: 학술 용도 라이선스 신청 필수 (승인 1–3 일)
3. **diff-gaussian-rasterization 빌드**: GCC 버전, CUDA_HOME 확인

Appendix A

트러블슈팅

A.1 흔한 설치 오류와 해결

Table A.1: 설치 트러블슈팅

오류	원인	해결
`RuntimeError: CUDA out of memory`	GPU VRAM 부족	배치 크기 축소, 해상도 다운샘플링, gradient checkpointing 활성화
`No CUDA GPUs are available`	드라이버 미설치 또는 버전 불일치	`nvidia-smi` 확인, 드라이버 재설치
`nvcc fatal: Unsupported gpu architecture`	CUDA_ARCHITECTURES 미설정	`cmake -DCMAKE_CUDA_ARCHITECTURES=native`
diff-rasterizer 빌드 실패	GCC 버전 비호환	`sudo apt install gcc-11 g++-11; CC=gcc-11 CXX=g++-11 pip install ...`
`ModuleNotFoundError: smplx`	SMPL-X 모델 경로 미설정	`export SMPLX_MODEL_PATH=~/models/smplx`
COLMAP SfM 실패 (0 images registered)	이미지 특징점 부족	빈 장면(피험자 없이) 촬영 후 보정; 텍스처 풍부한 장면 사용
OpenSim import 오류	conda 환경 충돌	별도 conda 환경 또는 `conda install` 재시도

Appendix B

참고 리포지토리 목록

Table B.1: 핵심 GitHub 리포지토리

도구	리포지토리
3D Gaussian Splatting	`graphdeco-inria/gaussian-splatting`
GART	`JiahuiLei/GART`
GauHuman	`skhu101/GauHuman`
ExAvatar	`mks0601/ExAvatar`
COLMAP	`colmap/colmap`
MMPose (ViTPose)	`open-mmlab/mmpose`
EasyMocap	`zju3dv/EasyMocap`
SMPL-X (smplx)	`vchoutas/smplx`
Pose2Sim	`perfanalytics/pose2sim`
FreeMoCap	`freemocap/freemocap`
OpenSim	`opensim-org/opensim-core`